

Házi feladat — Global Payment Service

Bankmonitor | hiring-2026-developer | 2026-08-08

Technikai feladat — Global Payment Service

Áttekintés

Egy új generációs fizetési átjárót építünk. A feladatod egy **működő fullstack alkalmazás** elkészítése: egy Spring Boot backend, amely felhasználói számlákat kezel és utalásokat dolgoz fel közöttük, valamint egy React frontend, amelyen ez a szolgáltatás használható.

Mit értékelünk

Nem a funkciólista teljes kipipálását keressük, hanem a **mérnöki döntéseidet**: hogyan bontod fel a problémát, hol húzod meg a határokat a rétegek között, mit hagysz ki tudatosan, és mindezt hogyan dokumentálod.

Irányadó ráfordítás: kb. 10–12 óra. Ez szándékosan kevesebb annál, mint amennyi minden szempont alapos kidolgozásához kellene. Amit nem valósítasz meg, azt írd le TODO-ként a README-ben — azzal együtt, hogy miért ezt a sorrendet választottad. **A TODO-listádat ugyanolyan súllyal értékeljük, mint a megírt kódot.**

Tech stack

Backend

- **Nyelv:** Java 21+
- **Framework:** Spring Boot 4.x
- **Adatbázis:** H2 (in-memory) vagy bármely SQL-alapú adatbázis
- **Build tool:** Maven vagy Gradle

Frontend

- **Kötelező:** React + TypeScript
 - **Minden más a te döntésed:** meta-framework, komponenskönyvtár, state-kezelés, styling, build tool, tesztelési eszközök. A README-ben indokold a választásaidat.
-

Funkcionális követelmények

1. Számla- és utalási logika

- **Számla létrehozása:** Felhasználói számla létrehozása egyenleggel és devizanemmel (EUR, USD, HUF).
- **Pénzutalás:** `POST /api/transfers`
 - Utalás két számla között.
 - Eltérő devizanem esetén az árfolyamot egy mockolt külső API-ból kell lekérni.

- **Megkötés:** A külső API „flaky” (503-ak, késleltetés). Kezeld ezt elegánsan.
- **Tranzakciók lekérdezése:** A végrehajtott utalások legyenek lekérdezhetők API-n keresztül.

2. Kötelező: idempotencia

- Minden utalási kérésnek tartalmaznia **kell** egy *X-Idempotency-Key* headert.
- Ha ugyanaz a kulcs kétszer érkezik meg azonos payloaddal:
 - Ha az első kérés sikeres volt, add vissza az eredeti *201 Created* eredményt.
 - Ha az első kérés még feldolgozás alatt van, adj vissza *409 Conflict*-ot (vagy más megfelelő státuszt).
 - Ha az első kérés hibára futott, a felhasználó újrapróbálhatja.
- **Cél:** Egy hálózati újrapróbálkozás soha ne eredményezzen dupla terhelést.

3. Rendszerintegráció

- A fizetési szolgáltatás egy nagyobb architektúra része. Más domain-szolgáltatásoknak (pl. Fraud Detection, Notification Center) tudniuk kell minden sikeres utalásról.
- **Cél:** Valósítsd meg ennek az információnak a továbbítását a külvilág felé.

4. Frontend alkalmazás

Készíts egy React alkalmazást, amelyen keresztül a fenti szolgáltatás használható. Három képernyőre van szükség:

1. **Számlák** — a meglévő számlák megjelenítése és új számla létrehozása.
2. **Utalás** — utalás indítása két számla között, összeg és devizanem megadásával, a művelet eredményének visszajelzésével.
3. **Tranzakciók** — a végrehajtott utalások listázása.

Ezen a három képességen túl a képernyők felépítése, a felhasználói élmény, a megjelenítés és a kód szerkezete a te terved szerint alakul. A README-ben írd le, mit tartottál fontosnak és miért.

Nem-funkcionális követelmények

1. Konkurencia és adatintegritás

Gondold végig, hogyan viselkedik a rendszer terhelés alatt. Hogyan kezeled, ha több kérés egyszerre érinti ugyanazt a számlát vagy ugyanazt az idempotencia-kulcsot?

2. Tesztelés

Mutasd meg a tesztelési megközelítésedet a **teljes stacken**. Az érdekel minket, hogyan igazolod a megoldásod helyességét és megbízhatóságát a különböző szinteken és forgatókönyvekben.

3. AI-használat

Használhatsz AI-eszközöket (Claude, ChatGPT, Copilot, Cursor stb.) a munkához.

- **Feltétel:** Ha használtad, csatolj egy *PROMPTS.md* fájlt.
- **Tartalom:**

- A lényeges promptok, amelyekkel kódot, teszteket vagy architekturális ötleteket generáltál — kiegészítve azzal, hol fogadtad el, hol javítottad ki és hol dobtad el az AI javaslatát.
- **Milyen eszközkészletet építettél magad köré:** ha használtál skilleket, saját agenteket/subagenteket, MCP szervereket, slash parancsokat, rules- vagy CLAUDE.md/AGENTS.md-szerű fájlokat, sorold fel őket, és írd le röviden, mire használtad melyiket. Ha van saját, korábban összeállított készleted, azt is említsd meg — nem kell megosztanod a tartalmát, elég, ha leírod, mit csinál.
- Ha van olyan konfiguráció vagy segédfájl a repóban, ami az AI-munkafolyamatodat szolgálja, azt hagyd is benne — nem zavaró, hanem jelzésértékű.

Ezt a fájlt **önálló értékelési szempontként** kezeljük: nem az számít, mennyit használtad az AI-t, hanem hogy mennyire tartottad kézben, és mennyire tudatosan építetted fel köré a munkádat.

Beadás

4. **Repository:** Git repository linkje vagy ZIP fájl.

5. **README.md:** Fejtsd ki benne az alábbiakat:

- **Architektúra és döntések:** Milyen architekturális és technológiai döntéseket hoztál a backend és a frontend oldalon, és miért? Mi mellett döntöttél, és mit vetettél el?
- **Hogyan álltál neki:** Mielőtt az első sort megírtad, készítettél-e tervet? Ha igen, milyen bontásban és hol vezetted? Melyik réteggel vagy komponenssel kezdtél, és miért azzal?
- **Edge case-ek:** Hogyan kezelted a feladatban említett eseteket (reziliencia, konkurencia, megbízhatóság)?
- **TODO-lista:** Mit nem valósítottál meg, miért, és milyen sorrendben folytatnád?
- **Éles üzem:** Mi kell ahhoz, hogy ez a rendszer valódi ügyfelekkel, éles környezetben is működhessen? Ha nem 10–12 órád lenne rá, hanem egy teljes sprinted, mivel folytatnád?
- **Futtatás:** Hogyan lehet buildelni, futtatni és tesztelni az alkalmazást.

6. **PROMPTS.md:** Az AI-használatod dokumentációja (ha volt).

Kérdések

Ha bármi nem egyértelmű a feladatban, írd bátran az interview@bankmonitor.hu címre — a kérdéseidre igyekszünk gyorsan válaszolni.

Miről fogunk beszélgetni

A személyes interjún a beadott megoldásodból indulunk ki: a döntéseidről, a tudatosan kihagyott részekről, a TODO-listádról és arról, hogyan vinnéd tovább a rendszert éles üzemig. Készülj rá, hogy a saját kódodat mind backend, mind frontend oldalon meg fogod tudni védeni.